

DSAA2043: Exercise 2

Notes

- This exercise is for practice only and does not count toward the final score.
- Both 0-indexed and 1-indexed arrays are accepted. However, when using Python, array indices must start from 0.

Problem 1. Merge Sort exemplifies the divide-and-conquer paradigm common in recursive algorithms: it breaks the sorting problem into smaller subproblems by recursively dividing the array in half, solves these simpler cases (single elements are inherently sorted), and then efficiently combines solutions by merging sorted subarrays in linear time to produce a completely sorted sequence. Figure 1 demonstrates how Merge Sort works.

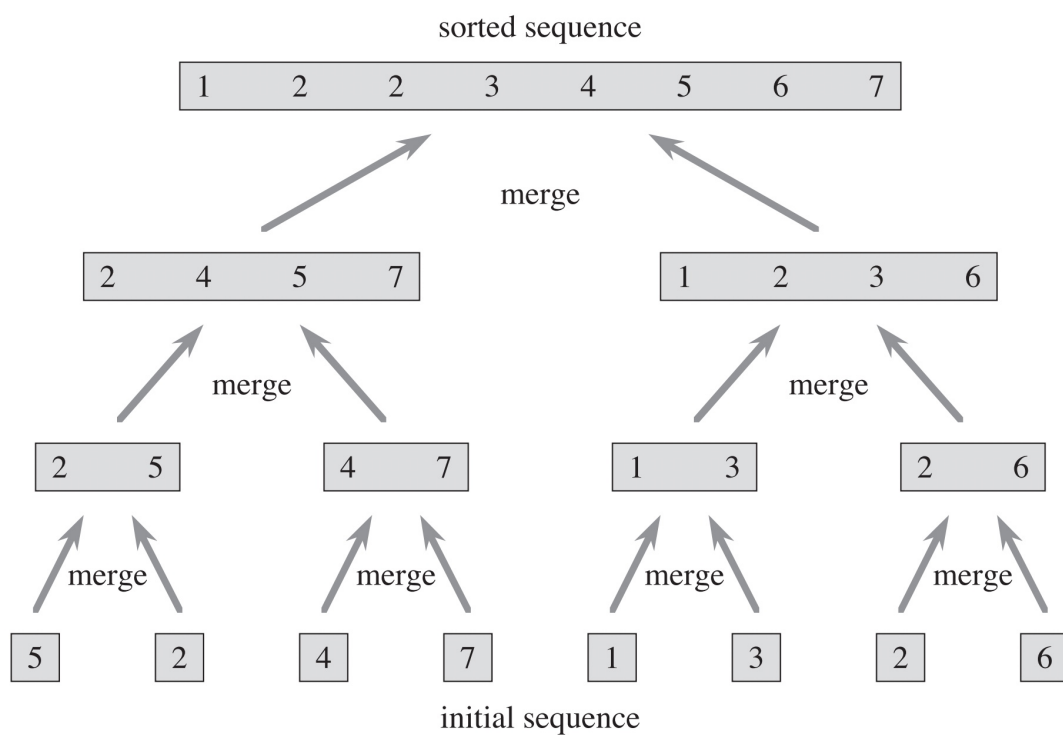


Figure 1: The operation of Merge-Sort on the array $A = \langle 5, 2, 4, 6, 1, 3, 2, 6 \rangle$. The lengths of the sorted sequences being merged increase as the algorithm progresses from bottom to top.

Algorithm 1 gives a recursive implementation of Merge Sort and the recurrence for the worst-case running time $T(n)$ of merge sort is:

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1, \\ 2T(n/2) + \Theta(n), & \text{if } n > 1. \end{cases}$$

Answer the following questions:

Q1.1. Using Figure 1 as a reference, illustrate the step-by-step execution of Merge Sort on the array $A = \langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$.

Algorithm 1 MERGESORT(A, p, r)

```
1: if  $p < r$  then
2:    $q = \lfloor (p + r) / 2 \rfloor$ 
3:   MERGESORT( $A, p, q$ )
4:   MERGESORT( $A, q + 1, r$ )
5:   MERGE( $A, p, q, r$ )
6: end if
```

Q1.2. Provide the code or pseudocode for the iterative implementation of the merge sort algorithm (meaning that the algorithm is implemented without recursion).

Answer 1.1. Below is the step-by-step execution of Merge Sort on the given array:

Index	0	1	2	3	4	5	6	7
Initial	5	2	4	6	1	3	2	6
Divide-1	5	2	4	6	1	3	2	6
Divide-2	5	2	4	6	1	3	2	6
Divide-3	5	2	4	6	1	3	2	6
Divide-4	5	2	4	6	1	3	2	6
Merge-1	2	5	4	6	1	3	2	6
Merge-2	2	4	5	6	1	2	3	6
Merge-3	1	2	2	3	4	5	6	6

Answer 1.2. Below is the pseudocode.

Algorithm 2 Iterative MERGESORT(A, n)

```
1:  $width \leftarrow 1$ 
2: while  $width < n$  do
3:   for  $i \leftarrow 1$  to  $n$  by  $2 \cdot width$  do
4:      $p \leftarrow i$ 
5:      $q \leftarrow \min(i + width - 1, n)$ 
6:      $r \leftarrow \min(i + 2 \cdot width - 1, n)$ 
7:     MERGE( $A, p, q, r$ )
8:   end for
9:    $width \leftarrow 2 \cdot width$ 
10: end while
```

Problem 2. Let S_1 be a set of n integers, and S_2 another set of n integers. Each of S_1 and S_2 is stored in an array of length n . The arrays are not necessarily sorted. Design an algorithm to determine whether $S_1 \cap S_2$ is empty. Your algorithm must terminate in $O(n \log n)$ time.

Answer 2. We solve this problem by combining the two input sets and sorting them as described in Problem 2. The steps are as follows:

1. **Combine the arrays:** Construct a new array A of size $2n$, by combining S_1 and S_2 . Each element is represented as a pair (v, t) , where:
 - v is the integer value.
 - t is the source identifier: $t = 1$ if the element comes from S_1 , and $t = 2$ if it comes from S_2 .
2. **Sort the combined array:** Sort A using the algorithm from Problem 2. The comparison of two elements $e_1 = (v_1, t_1)$ and $e_2 = (v_2, t_2)$ is defined as:
 - If $v_1 < v_2$, then $e_1 < e_2$.
 - If $v_1 > v_2$, then $e_1 > e_2$.
 - If $v_1 = v_2$, then compare their source identifiers:
 - If $t_1 < t_2$, then $e_1 < e_2$.
 - Otherwise, $e_1 > e_2$.

Sorting A takes $O(n \log n)$ time.

3. **Scan the sorted array:** Traverse the sorted array A , and for each consecutive pair of elements $e_1 = (v_1, t_1)$ and $e_2 = (v_2, t_2)$:
 - If $v_1 = v_2$ and $t_1 \neq t_2$, then $S_1 \cap S_2 \neq \emptyset$. Return **true**.
 - Otherwise, continue scanning.

Scanning A takes $O(n)$ time.

If no matching pair (v_1, t_1) and (v_2, t_2) is found, return **false**.

Complexity Analysis:

- Constructing A takes $O(n)$ time.
- Sorting A takes $O(n \log n)$ time.
- Scanning A takes $O(n)$ time.

Thus, the total time complexity is $O(n \log n)$, as required.

Problem 3. Given an array $A = \langle 46, 74, 53, 14, 26, 38, 86, 65, 27, 34 \rangle$, use the quicksort method with the first element as the pivot. Illustrate the step-by-step execution and the final result. The execution result of the first partition is given as follows:

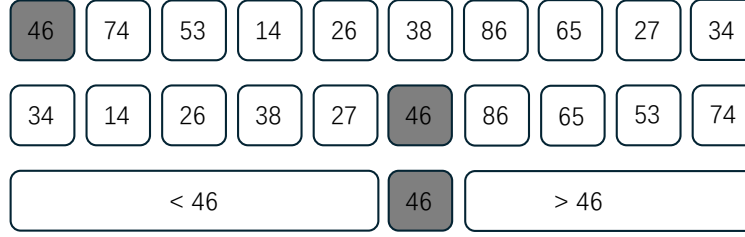


Figure 2: The execution result of the first partition on array $A = \langle 46, 74, 53, 14, 26, 38, 86, 65, 27, 34 \rangle$

Answer 3. Below is the step-by-step execution of QuickSort on the given array:

Index	0	1	2	3	4	5	6	7	8	9
Initial	46	74	53	14	26	38	86	65	27	34
partition1	34	14	26	38	27	46	86	65	53	74
partition2	27	14	26	34	38	46	74	65	53	86
partition3	26	14	27	34	38	46	53	65	74	86
partition4	14	26	27	34	38	46	53	65	74	86
partition5	14	26	27	34	38	46	53	65	74	86

Problem 4 (2-Sum-Problem). Given an unsorted array A of integers and a target value v , design an algorithm to determine whether A contains two different integers x and y such that $x + y = v$. The algorithm must run in $O(n \log n)$ time.

Answer 4. To solve this problem efficiently, we use **binary search**. For each element x in the array, we search for $v - x$ using binary search. The algorithm proceeds as follows:

1. Iterate through each element x in the array A (indexed as $A[i]$).
2. Compute the target value $t = v - x$.
3. Use binary search to check whether t exists in A , ensuring that the index of t is different from the index of x (i.e., avoid using the same element twice).
4. If t is found, return **true**.
5. If the iteration completes without finding such a pair, return **false**.

The pseudocode for the solution is as follows:

```
def two_sum(A, v):
    for i in range(len(A)):
        x = A[i]
        t = v - x
        if binary_search(A, t, exclude_index=i):
            return True
    return False

def binary_search(A, target, exclude_index):
    low, high = 0, len(A) - 1
```

```

while low <= high:
    mid = (low + high) // 2
    if A[mid] == target and mid != exclude_index:
        return True
    elif A[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
return False

```

Time Complexity:

- The outer loop runs $O(n)$ times, once for each element in the array.
- Each binary search operation runs in $O(\log n)$.
- Therefore, the total time complexity of the algorithm is $O(n \log n)$.

Problem 5. Consider a set S of n integers that are stored in an array A (not necessarily sorted). Let e and e' be two integers in S such that e is positioned before e' in A . We call the pair (e, e') an **inversion in S** if $e > e'$.

Q5.1. Given $S = [1, 5, 3, 2, 6]$, calculate the total number of inversions in S .

Q5.2. Design an algorithm to count the number of inversions in S . The algorithm must run in $O(n \log n)$ time.

Answer 5.1. We iterate through all possible pairs of elements in S and count the number of inversions:

- Compare 1 with 5, 3, 2, 6: No inversions since $1 < 5$, $1 < 3$, $1 < 2$, $1 < 6$.
- Compare 5 with 3, 2, 6:
 - $5 > 3 \Rightarrow$ inversion $(5, 3)$.
 - $5 > 2 \Rightarrow$ inversion $(5, 2)$.
 - $5 < 6 \Rightarrow$ no inversion.
- Compare 3 with 2, 6:
 - $3 > 2 \Rightarrow$ inversion $(3, 2)$.
 - $3 < 6 \Rightarrow$ no inversion.
- Compare 2 with 6: No inversion since $2 < 6$.

The inversions are:

$(5, 3), (5, 2), (3, 2)$.

Thus, the total number of inversions in S is 3.

Answer 5.2. If $n = 1$, simply return 0. If $n \geq 2$, we divide A into two halves:

- (i) the first half includes the first $\lfloor n/2 \rfloor$ elements, and
- (ii) the second includes the rest.

Let A_1 be the array corresponding to the first half, and A_2 be the array corresponding to the second. We count the number c_1 of inversions in A_1 recursively, and then count the number c_2 of inversions in A_2 recursively. We ensure that (i) when the execution returns from A_1 , the array A_1 has been sorted, and (ii) the same is true for A_2 .

We now count the number c_3 of such inversions (e, e') that $e \in A_1$ and $e' \in A_2$. This can be achieved in $O(n)$ time utilizing the fact that both A_1 and A_2 have been sorted. Initially, set i and j to 1, and c_3 to 0. Next, repeat the following until either $i > |A_1|$ or $j > |A_2|$:

- If $A_1[i] > A_2[j]$, then increase c_3 by $\lfloor n/2 \rfloor - i + 1$, and increase j by 1;
- Otherwise, increase i by 1.

The total number of inversions equals $c_1 + c_2 + c_3$.

Before returning to the upper level of recursion, we merge A_1 and A_2 into one sorted list A' , and copy the elements of A' into A (which thus becomes sorted). This takes $O(n)$ time.

Let $f(n)$ be the worst-case running time of our algorithm. It holds that $f(1) = O(1)$, and

$$f(n) = 2 \cdot f(\lfloor n/2 \rfloor) + O(n).$$

By the master theorem, we have $f(n) = O(n \log n)$.

Problem 6. Let $f(n)$ be a function of positive integer n . We know:

$$\begin{aligned} f(1) &= 1, \\ f(n) &\leq 2 + f(\lceil n/10 \rceil). \end{aligned}$$

Prove $f(n) = O(\log n)$ using the **expansion method**. Recall that $\lceil x \rceil$ is the ceiling operator that returns the smallest integer at least x .

Answer 6. Consider first n being a power of 10. We expand $f(n)$ as follows:

$$\begin{aligned} f(n) &\leq 2 + f\left(\frac{n}{10}\right) \\ &\leq 2 + 2 + f\left(\frac{n}{10^2}\right) \\ &\leq 2 + 2 + 2 + f\left(\frac{n}{10^3}\right) \\ &\dots \\ &\leq 2 \cdot \log_{10} n + f(1) \\ &= 2 \cdot \log_{10} n + 1 \\ &= O(\log n). \end{aligned}$$

Now consider n that is not a power of 10. Let n' be the smallest power of 10 that is greater than n . We have:

$$\begin{aligned} f(n) &\leq f(n') \\ &\leq 2 \cdot \log_{10} n' + 1 \\ &\leq 2 \cdot \log_{10}(10n) + 1 \\ &\leq O(\log n). \end{aligned}$$